Integration
TECHNOLOGIES

InterWeave IDE v2.4 User Guide

Date: 18 October 2007
Version 2.9
Integration Technologies Client Support and Engineering

© 2006 Integration Technologies, Inc.

Integration Technologies, Inc.

POWERED BY InterWeave Analyze. Configure. Deploy.

All rights reserved by Integration Technologies, Inc. No part of this publication may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopy, recording or any information storage and retrieval system, without permission in writing from Integration Technologies, Inc. This document contains proprietary information furnished for evaluation purposes only; except with the express written permission of Integration Technologies, Inc. ("Integration Technologies"), such information may not be published, disclosed, or used for any other purpose. You acknowledge and agree that this document and all portions thereof, including, but not limited to, any copyright, trade secret and other intellectual property rights relating thereto, are and at all times shall remain the sole property of Integration Technologies and that title and full ownership rights in the information contained herein and all portions thereof are reserved to and at all times shall remain with Integration Technologies. You acknowledge and agree that the information contained herein constitutes a valuable trade secret of Integration Technologies. You agree to use utmost care in protecting the proprietary and confidential nature of the information contained herein.

Document History

Version	Date	Author	Comment
1.0	25 Sep 2006	Dmytro Zotkin	Initial version
2.0	28 Sep 2006	Inna Levytska	Screenshots added
2.1	01.Oct.2006	Dmytro Zotkin	File Adaptor commands updated
2.2	05.Oct.2006	Dmytro Zotkin	Replace double quotes added
2.3	08.Oct.2006	Dmytro Zotkin	File Adaptor writing of multiple files
2.4	24.Oct.2006	Dmytro Zotkin	E-Mail Adaptor, Basic Authorization
2.5	30.Oct.2006	Inna Levytska	Input and Output parameters updated
2.6	06.Nov.2006	Dmytro Zotkin	FTP Adaptor added
2.7	20.Feb.2007	Inna Levytska	Asynchronous JDBC Adaptor added
2.8	23. Jun. 2007	Inna Levytska	List of global variables updated
2.9	18.Oct.2007	Inna Levytska	Various changes

1 IW IDE Syntax Elements 5
1.1 Parameters in IW IDE 5
1.1.1 Transaction Flow Parameters 5
1.1.2 Data Map Parameters 7
1.1.2.1 General Parameters 10
1.1.2.1.1 Mapping type <empty> 10
1.1.2.1.2 Mapping type “return” 11
1.1.2.1.2.1 Mapping value <empty> 12
1.1.2.1.2.2 Mapping value “ms” 12
1.1.2.1.2.3 Mapping value “s” 13
1.1.2.1.2.4 Mapping value ”z” 13
1.1.2.1.2.5 Mapping value ”dpu” 14
1.1.2.1.2.6 Mapping value ”tpu” 14
1.1.2.1.2.7 Mapping value ”dou” 15
1.1.2.1.2.8 Values for internal usage 15
1.1.2.1.3 Mapping types “replacequote”, “replacedoublequote” 16
1.1.2.1.4 Mapping type “filter” 17
1.1.2.1.5 Mapping type “space_substitute” 18
1.1.2.1.6 Mapping type “xpath” 18
1.1.2.1.7 Reserved Mapping types 19
1.1.2.1.8 Mapping type “transform” 20
1.1.2.1.9 Mapping type “constant” 20
1.1.2.1.10 Mapping type “default” 21
1.1.2.2 Input Parameters 23
1.1.2.2.1 Method type <java type> 23
1.1.2.2.2 Method type “XML” 24
1.1.2.3 Output Parameters 25
1.1.2.3.1 Method Type <empty> 25
1.1.2.3.2 Method Type <java type> 26
1.1.2.3.3 Method Type “XML” 27
1.1.2.3.4 Method Type “xpath” 27
1.1.2.3.5 Method Type “reflect” 28
1.2 Dynamic Connection parameters 29
1.2.1 Reference to Transaction Flow Parameter 29
1.2.2 Reference to Data Map parameter 31
1.3 Dynamic command parameters 31
1.3.1 Substitute parameters 31
1.3.2 Global variables 33
1.3.3 Transaction Flow Counter 35
2 IW Adaptors 36
2.1 JDBC Adaptor 36
2.1.1 Asynchronous JDBC Adaptor 36
2.1.2 Synchronous JDBC Adaptor 36
2.2 HTTP Adaptor 36
2.3 XML over HTTP Adaptors 38
2.3.1 Base XML Adaptor 38

2.3.2 Hierarchical XML adaptor 39
2.4 SOAP over HTTP Adaptors 39
2.4.1 Base SOAP Adaptor 40
2.4.2 Hierarchical SOAP adaptor 40
2.5 File Adaptors 40
2.5.1 Base File Adaptor 42
2.5.2 Base FTP Adaptor 42
2.6 API Adaptors 43
2.6.1 Local API Adaptor 43
2.7 E-mail Adaptors 43
2.7.1 Base E-mail Adaptor 44

1 IW IDE Syntax Elements

1.1 Parameters in IW IDE

There are 2 types of parameters that are used for building transactions and transaction flows – Transaction Flow Parameters and Data Map Parameters.

1.1.1 Transaction Flow Parameters

These parameters are used to configure execution of the whole flow. They can be used directly in connection elements (see below 1.2.1) and indirectly in Data Maps (see below 1.1.2). Each parameter has 3 elements (see Figure 1):

- **Name** – the name of the parameter used to refer to it;
- **Value** – the initial (default) value of the parameter defined at design time;
- **Access Flag** – determines whether parameter is changeable during run-time via WEB interface

Access Flag can have 2 values – "Allowed" and "Prohibited". If "Allowed", then this parameter can be viewed and modified during run-time via WEB interface, otherwise the default value is "fixed" and parameter can not be viewed via WEB interface.

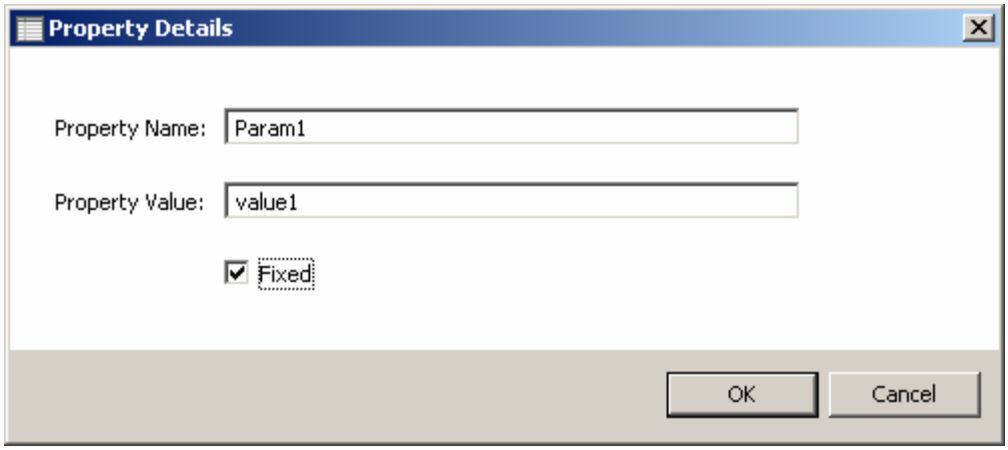

Figure 1

There are 3 built-in transaction flow parameters that should be a part of every transaction flow definition (see Figure 2):

Name	Value	Description
applicationname	The only supported value is "iwtransformationserver"	This parameter is used internally and must be "fixed" (can not be changeable via WEB interface)
tranname	The name of the first transaction in the flow	This parameter is strongly recommended to be "fixed" in production environment but can be changeable for testing purposes.
QueryStartTime	Initial timestamp	This parameter is normally used in scheduled transactions as a timestamp to query elements created or modified after it. In most cases should not be "fixed". There is a mechanism (see below 1.1.2.1) to return a new value for this parameter after transaction flow execution that will be the latest time of elements created/modified. This new value will be used by the next execution of the transaction flow. Also this parameter can be used as a regular timestamp for ETL or utility transactions.

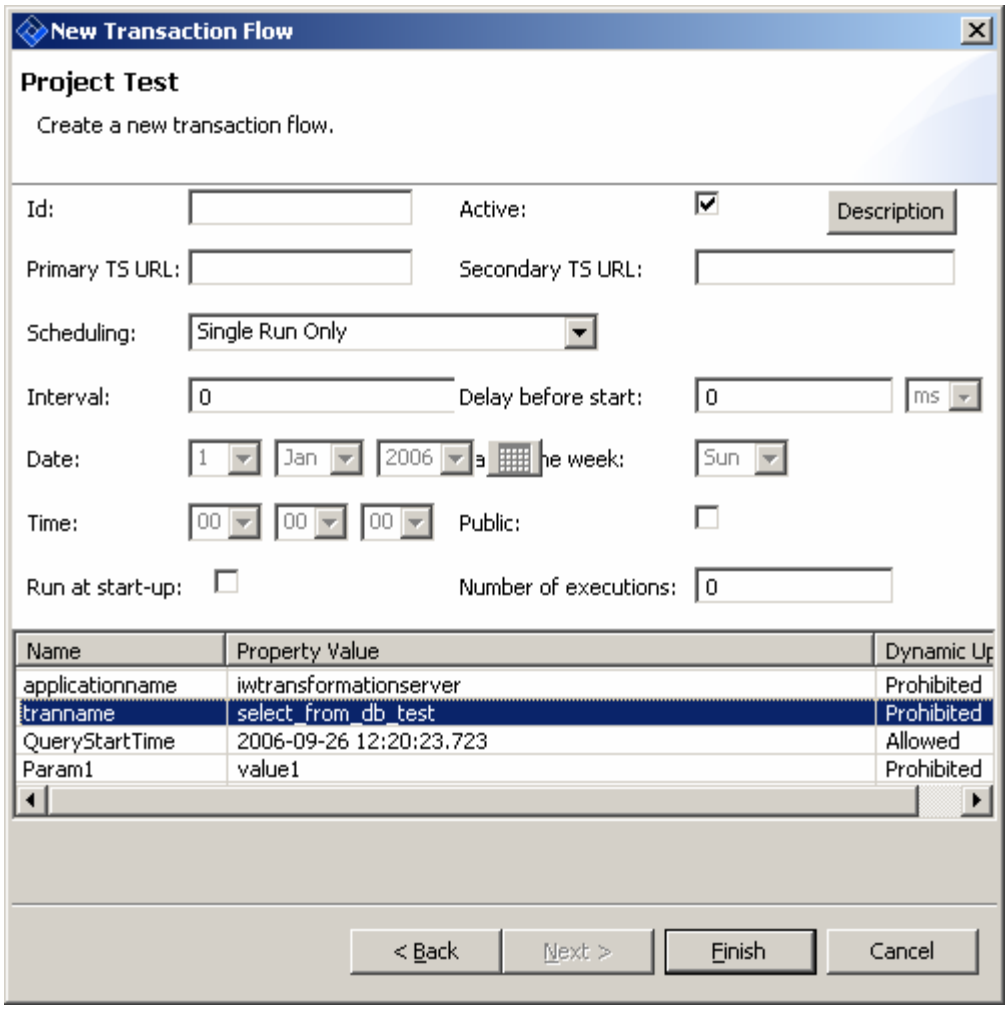

Figure 2

1.1.2 Data Map Parameters

These parameters are used to configure execution of the enclosing Data Map. They can be used directly in Data Map command as substitute parameters (see below 1.3.1) and indirectly to control command execution and process or modify its results. There are three types of Data Map Parameters (see Figure 3):

- **General Parameters** – for general usage
- **Input Parameters** – used by LocalAPIAdaptor to define method input parameters
- **Output Parameters** – generally used to dynamically create a global transaction flow parameters that can be used in next transactions as General Parameters.

Also Output Parameters can be used by LocalAPIAdaptor to define method output type. These two usages are mutually exclusive.

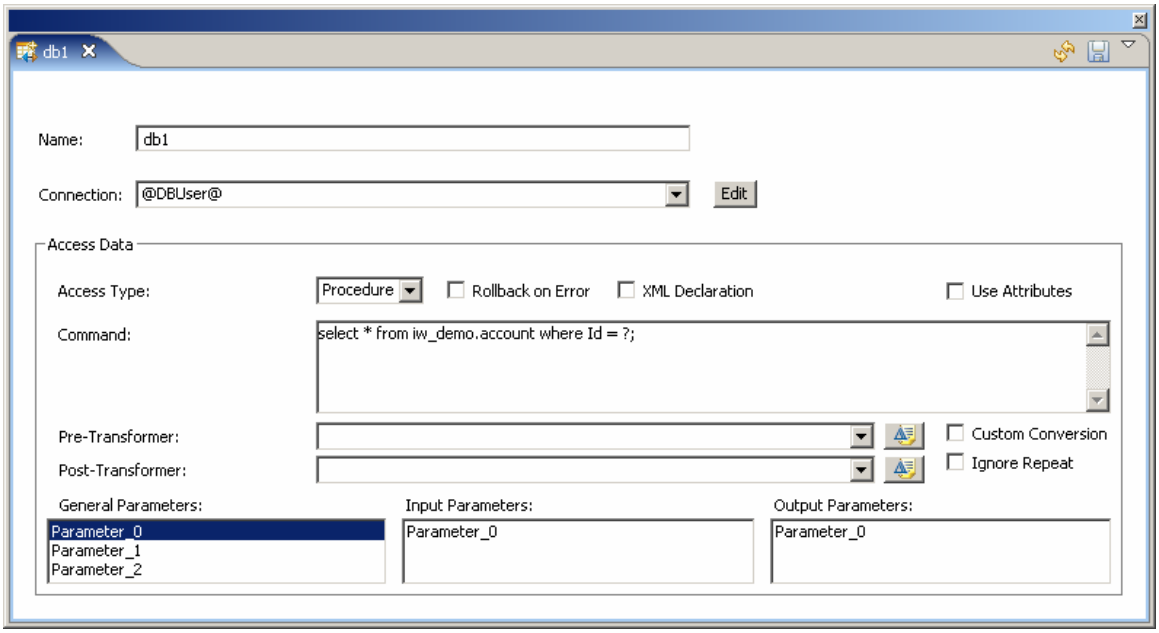

Figure 3

Regardless of type, each Data Map parameter has six elements (see Figure 4):

- **Input Name** – can be used as parameter name or for other purposes;
- **Mapping Type** – specifies the parameter type and behaviour – see below
- **Mapping Value** – contains some value; its meaning depends on Mapping Type – see below;
- **Quoted** – defines where the value must be surrounded by the quote characters (‘) when used; applicable to some Mapping Types only – see below
- **Pre-Transformer** – allows assigning to a parameter’s value the result of a transformation of previous transactions outcome if it is in HTML format; transformation will be performed by the transformer defined by this parameter. Applicable to an empty Mapping Type only.
- **Post-Transformer** – reserved for future use.

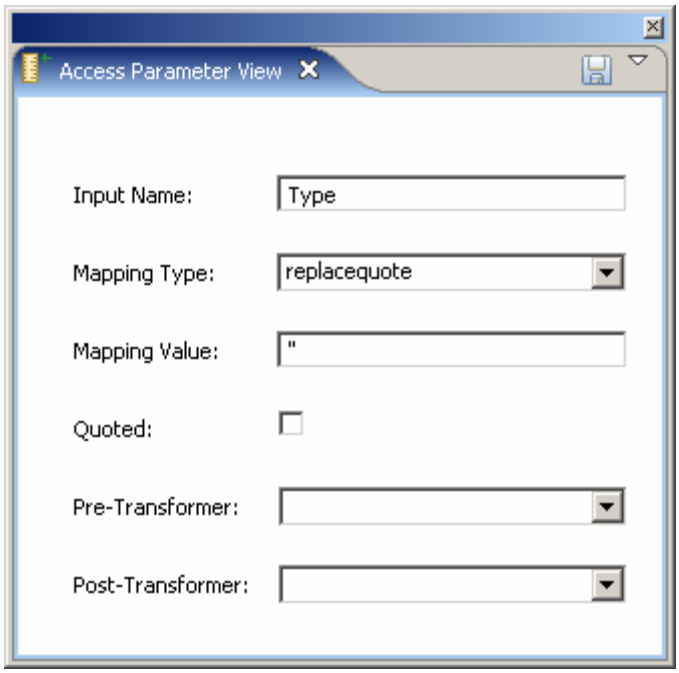

Figure 4

The following tables represent parameter usage depending on Parameter type and Mapping Type.

1.1.2.1 General Parameters

1.1.2.1.1 Mapping type <empty>

Input Name	Parameter Name
Mapping value	Ignored
Quoted	Used
Parameter value	The value from global flow parameter with the same name; this could be static flow parameter (see 1.1.1) or dynamic flow parameter (see 0). Generated before command execution (seeFigure 5).

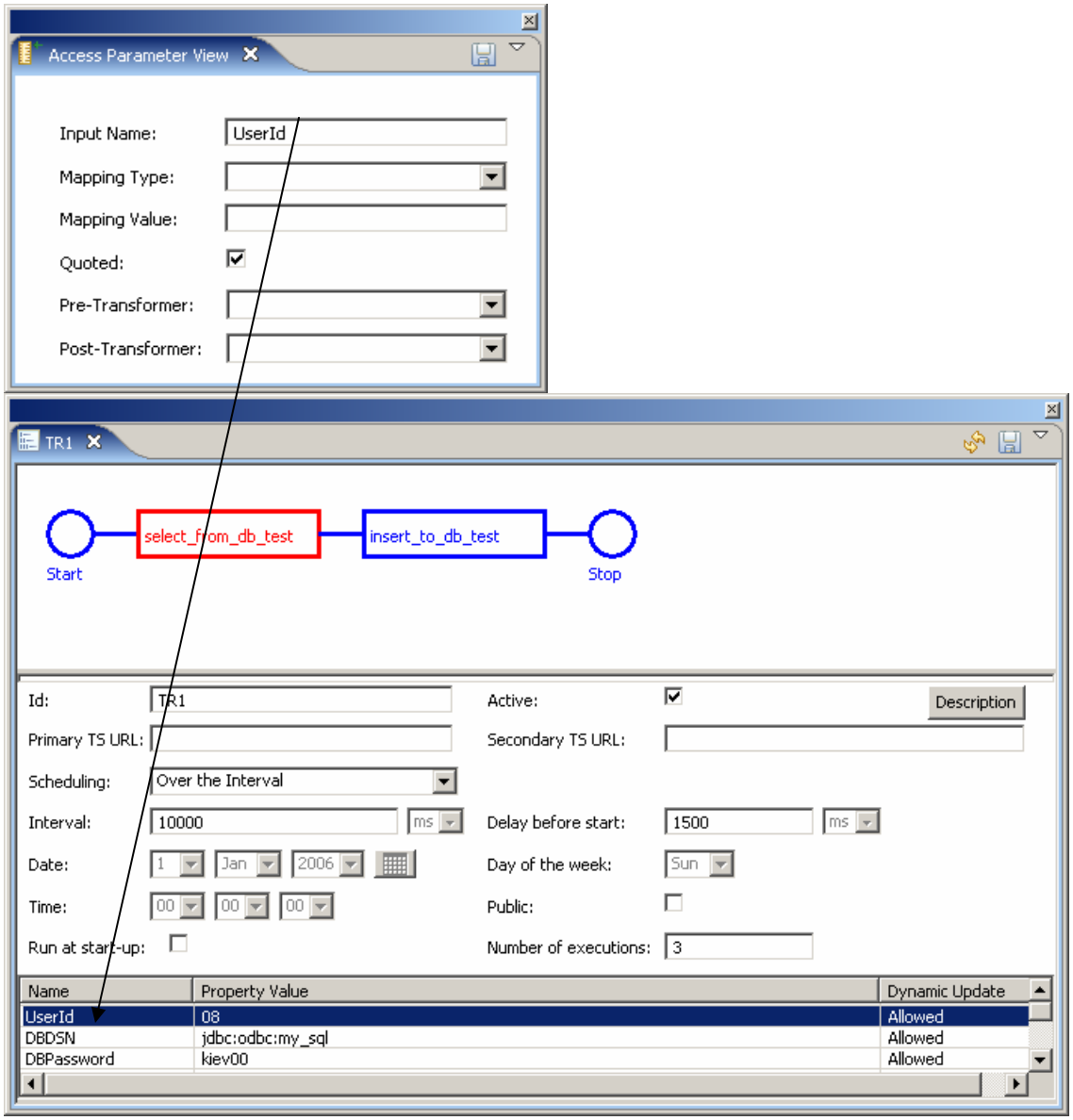

Figure 5

1.1.2.1.2 Mapping type "return"

Input Name	Name or name presentation of the field in the result data that contains return timestamp. If result is in IWP then just name should be used. If result is in HIWP, then path-like expression from rowset names will be used.
Quoted	N/A
Description	Value converted to JDBC escape format will be returned in QueryStartTime parameter. If field contains just date or time part of the timestamp, that part will be set. Generated after command execution.

1.1.2.1.2.1 Mapping value <empty>

Parameter value	Field contains timestamp in JDBC escape format (seeFigure 6)

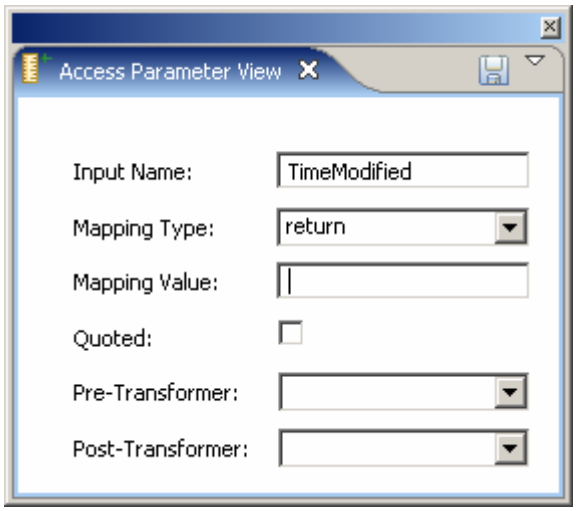

Figure 6

1.1.2.1.2.2 Mapping value "ms"

Parameter value	Field contains timestamp as UTC milliseconds from the epoch (see Figure 7).

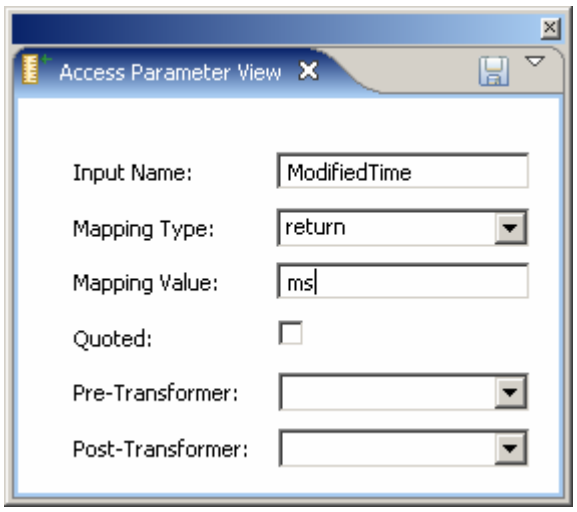

Figure 7

1.1.2.1.2.3 Mapping value “s”

Parameter value	Field contains timestamp as UTC seconds from the epoch (see Figure 8).

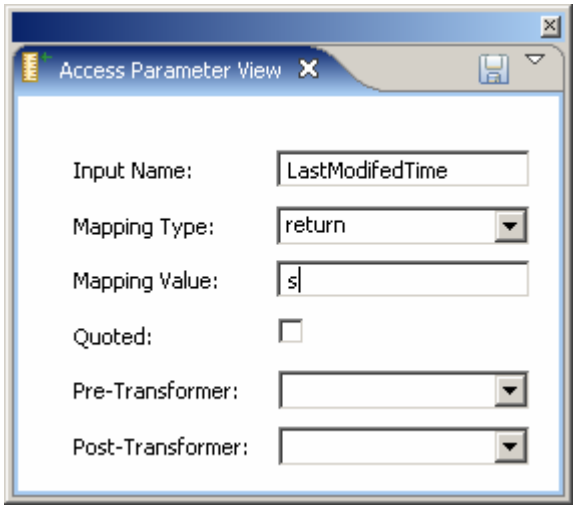

Figure 8

1.1.2.1.2.4 Mapping value ”z”

Parameter value	Field contains timestamp in ISO 8601 notation (seeFigure 9).

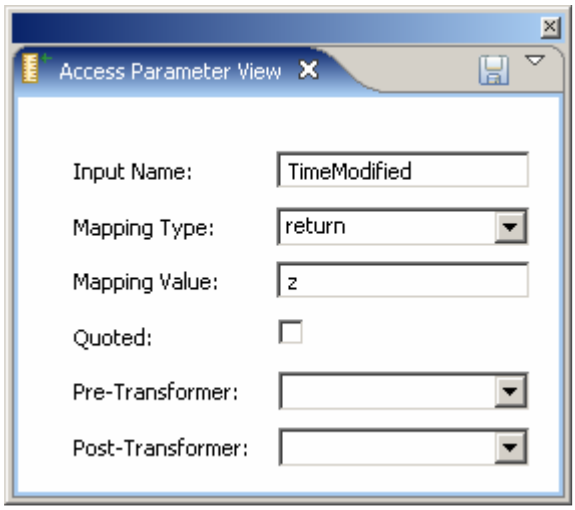

Figure 9

1.1.2.1.2.5 Mapping value ”dpu”

Parameter value	Field contains the date part of the timestamp in US type notation mm/dd/yyyy (seeFigure 10).

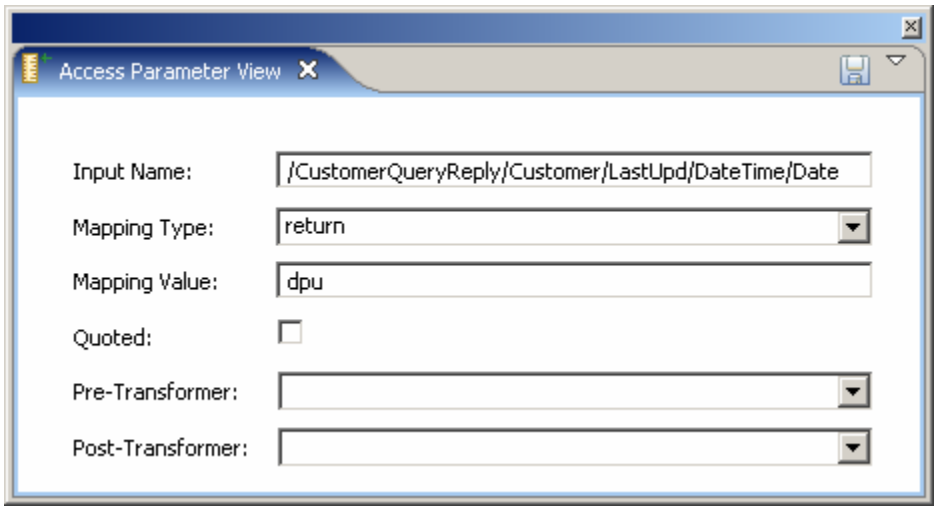

Figure 10

1.1.2.1.2.6 Mapping value ”tpu”

Parameter value	Field contains the time part of the timestamp in string notation hh:mm:ss (seeFigure 11).

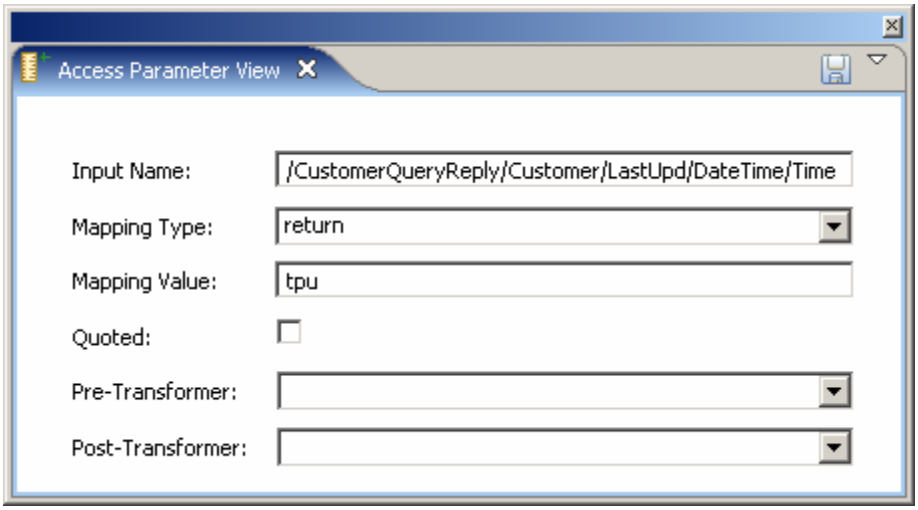

Figure 11

1.1.2.1.2.7 Mapping value ”dou”

Parameter value	Field contains the date part of timestamp in US type notation mm/dd/yyyy. Time part will not be available and set to 00:00:00.0 (seeFigure 12).

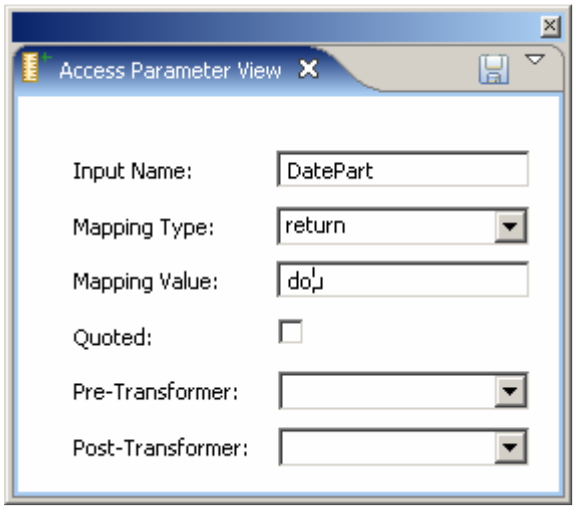

Figure 12

1.1.2.1.2.8 Values for internal usage

Mapping values ”p”, ”t”, and ”q” are reserved for internal usage.

1.1.2.1.3 Mapping types "replacequote", "replacedoublequote"

Input Name	Name or the name presentation of the field in the result data where quote character (') or double quote character (") will be replaced (seeFigure 13). Can contain star characters (*), in which case quotes in all fields will be replaced (see Figure 14).
Mapping value	Character to which quote or double quote will be replaced.
Quoted	N/A
Parameter value	Parameter of this type has no value. It controls execution by modifying execution results (replacing quotes or double quotes to specific characters).

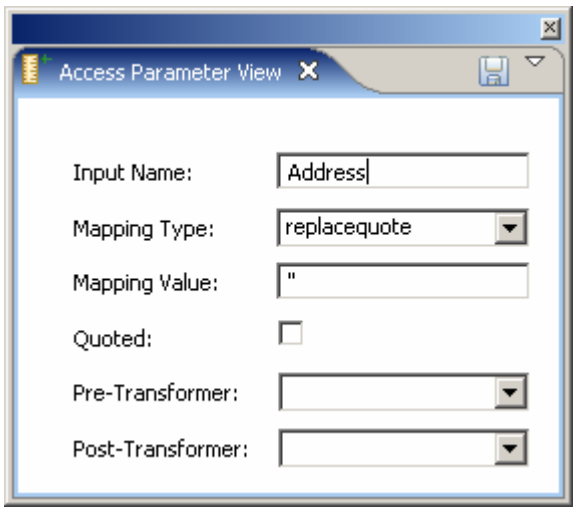

Figure 13

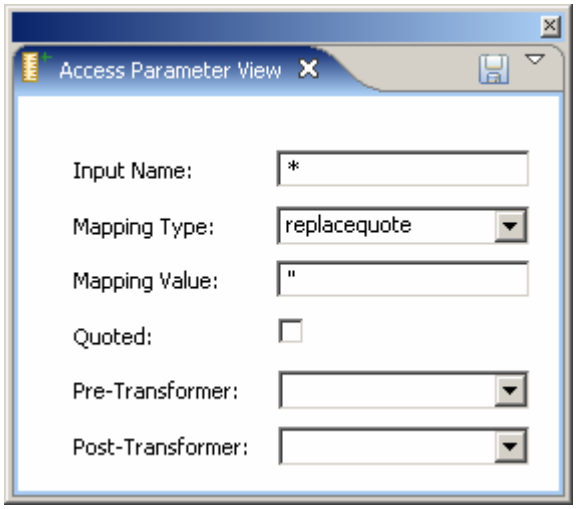

Figure 14

1.1.2.1.4 Mapping type "filter"

Input Name	Prefix to filter value (column name) used in filtering transactions (seeFigure 15).
Mapping value	Name of the column used as a filtering key.
Quoted	N/A
Parameter value	Parameter of this type has no value. It controls execution by filtering execution results. The result of the current transaction will contain only records where filtering keys do not match to those in filter transaction.

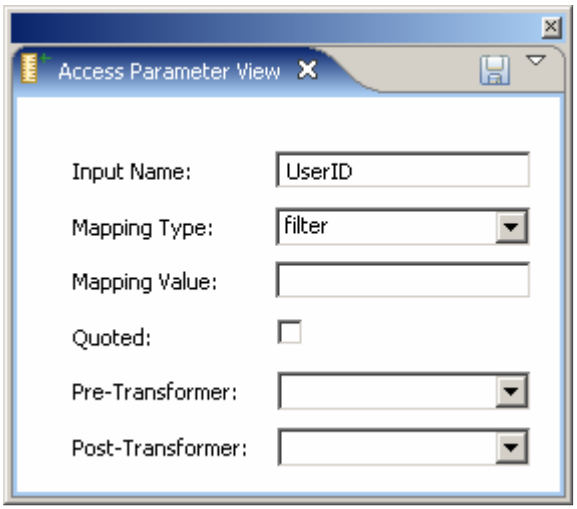

Figure 15

1.1.2.1.5 Mapping type "space_substitute"

Input Name	Parameter Name.
Mapping value	Character that will replace spaces.
Quoted	Used
Parameter value	The value from global flow parameter with the same name, in which all spaces are replaced with the specific character; this could be static flow parameter (see 1.1.1) or dynamic flow parameter (see 0). Generated before command execution (seeFigure 16).

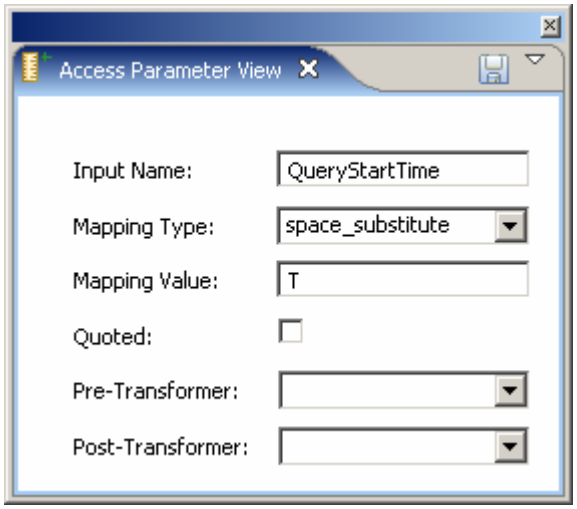

Figure 16

1.1.2.1.6 Mapping type "xpath"

Input Name	Parameter Name.
Mapping value	XPath expression to execute.
Quoted	Used
Parameter value	The result of XPath expression execution against previous transactions results. Generated before command execution (see Figure 17).

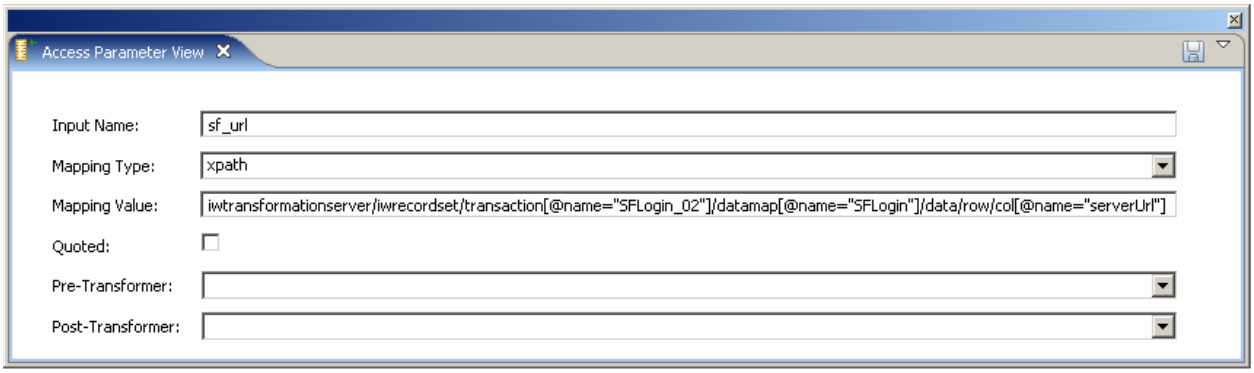

Figure 17

1.1.2.1.7 Reserved Mapping types

Mapping type “postxpath” and “post” are reserved for future use (in IWURLPost Adaptor).

1.1.2.1.8 Mapping type "transform"

Input Name	Name of the transformer.
Mapping value	N/A.
Quoted	Used
Parameter value	Result of a transformation of previous transactions outcome; transformation will be performed by the transformer defined by this parameter Input Name. Similar to pre-transformer functionality, but for any XML. Generated before command execution (see Figure 18).

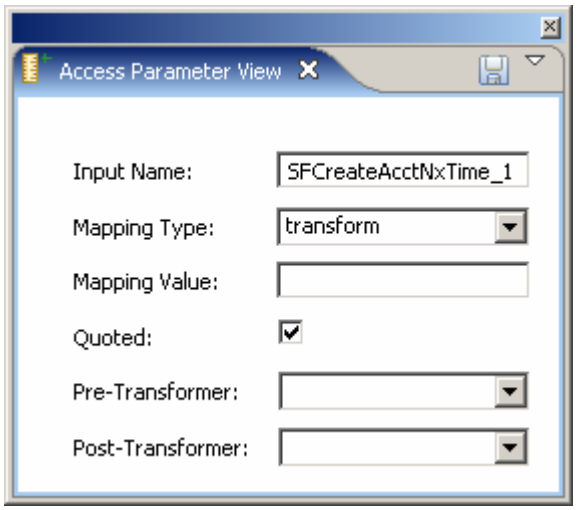

Figure 18

1.1.2.1.9 Mapping type "constant"

Input Name	Value of the parameter.
Mapping value	N/A.
Quoted	Used
Parameter value	Value of this parameter Input Name. Generated before command execution (seeFigure 19).

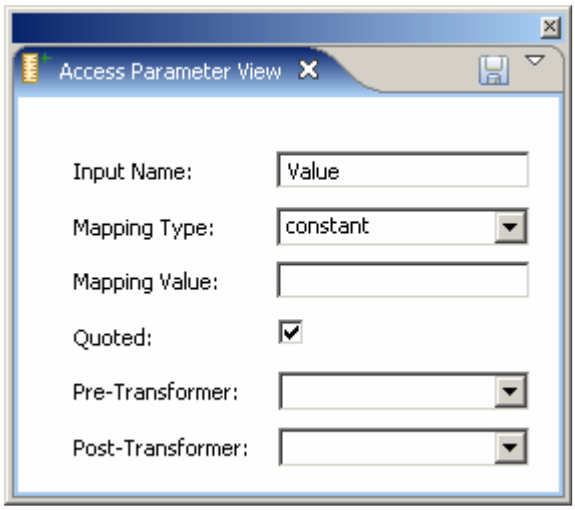

Figure 19

1.1.2.1.10 Mapping type "default"

Input Name	Parameter Name.
Mapping value	New parameter value for empty parameter.
Quoted	Used
Parameter value	The value from global flow parameter with the same name; this could be static flow parameter (see 1.1.1) or dynamic flow parameter (see 0). If this value is empty, the New Parameter Value (from Mapping Value) will be set as a value of this global parameter for the future transactions and used for this one. Generated before command execution (see Figure 20); This shown for clarification purposes only. No change of Parameter Value will happen during design time. It will happen during transaction flow execution).

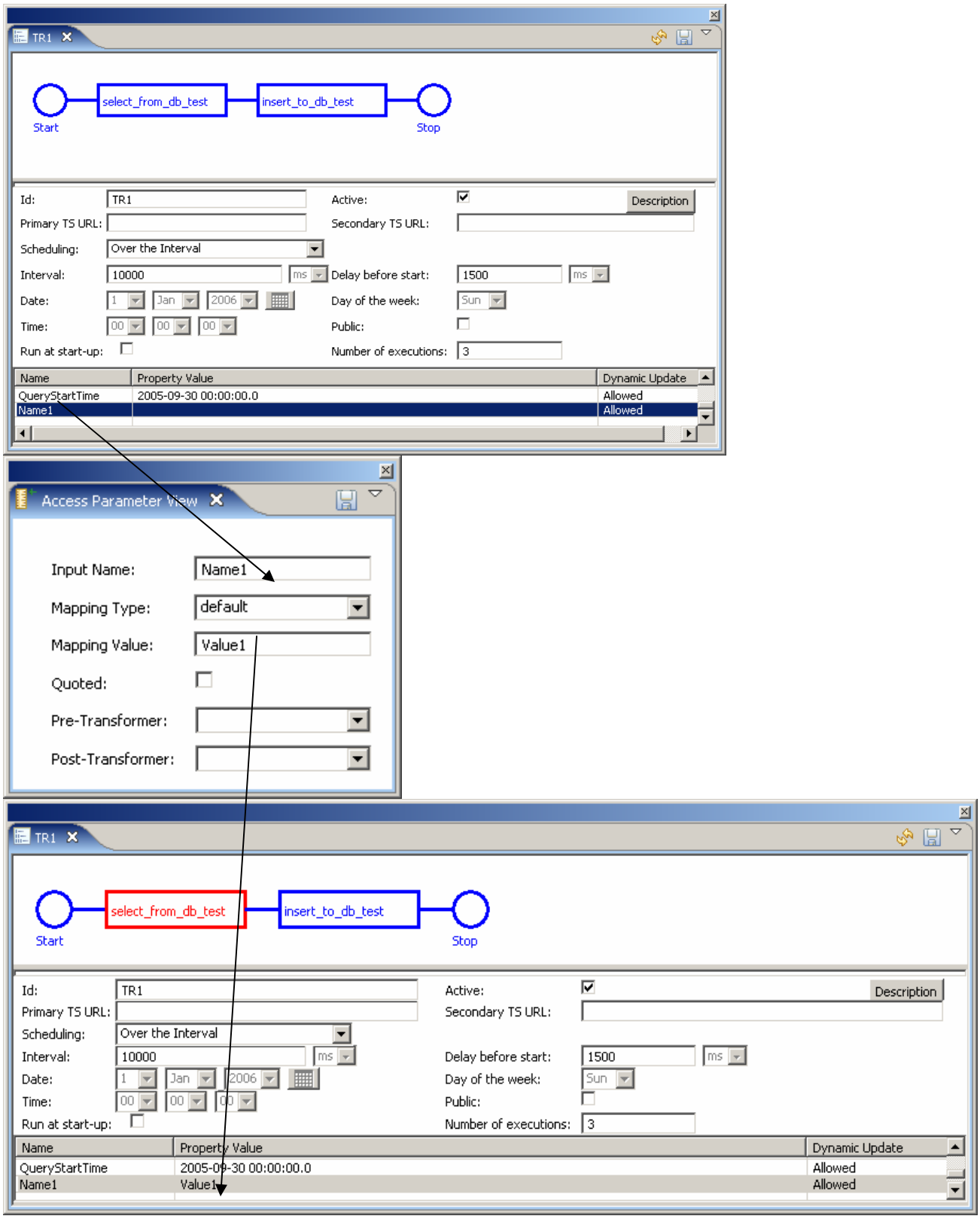
TR1
select_from_db_test
insert_to_db_test
Start
Stop
Id:
TR1
Active:
Description
Primary TS URL:
Secondary TS URL:
Scheduling:
Over the Interval
Interval:
10000
ms
Delay before start:
1500
ms
Date:
1
Jan
2006
Day of the week:
Sun
Time:
00
00
00
Public:
Run at start-up:
Number of executions:
3
Name
Property Value
Dynamic Update
QueryStartTime
2005-09-30 00:00:00.0
Allowed
Name1
Allowed
Access Parameter View
Input Name:
Name1
Mapping Type:
default
Mapping Value:
Value1
Quoted:
Pre-Transformer:
Post-Transformer:
TR1
select_from_db_test
insert_to_db_test
Start
Stop
Id:
TR1
Active:
Description
Primary TS URL:
Secondary TS URL:
Scheduling:
Over the Interval
Interval:
10000
ms
Delay before start:
1500
ms
Date:
1
Jan
2006
Day of the week:
Sun
Time:
00
00
00
Public:
Run at start-up:
Number of executions:
3
Name
Property Value
Dynamic Update
QueryStartTime
2005-09-30 00:00:00.0
Allowed
Name1
Value1
Allowed

Figure 20

1.1.2.2 Input Parameters

These parameters are used for API Adaptors only (see 2.6) to define Method Input parameters.

1.1.2.2.1 Method type <java type>

When parameter is created Mapping Type is replaced by Qualified Class Name (including package) of the method input parameter (see Figure 21).

Input Name	Method Name.
Mapping value	Sequence number (0-based) of the method input parameter.
Quoted	N/A
Parameter Value	Value of the input parameter of the method

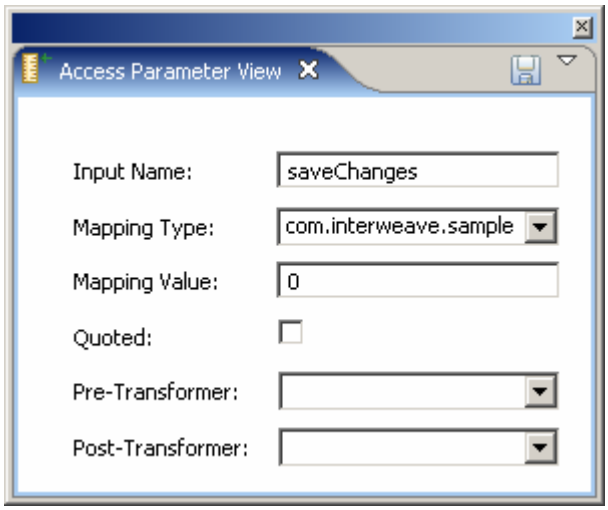

Figure 21

1.1.2.2.2 Method type "XML"

Input Name	Method Name
Mapping value	Sequence number (0-based) of the method input parameter. Only 0 (zero) is supported
Quoted	N/A
Parameter Value	XML string as only parameter of the method (see Figure 22)

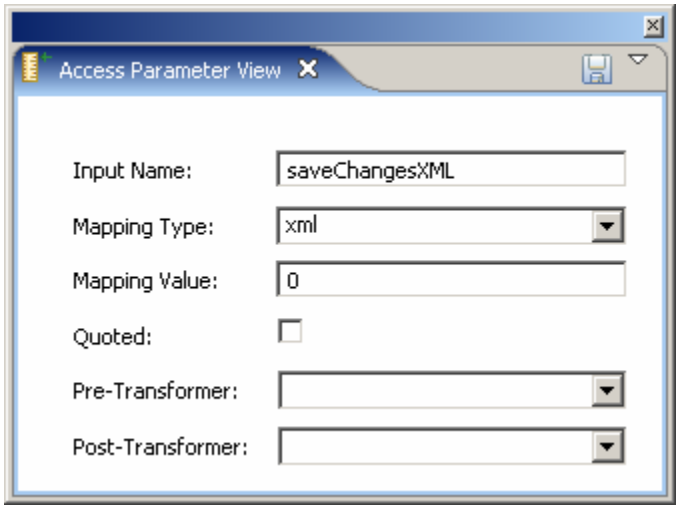

Figure 22

1.1.2.3 Output Parameters

1.1.2.3.1 Method Type <empty>

Input Name	Method Name
Mapping value	Name for the IWP “col” element.
Quoted	N/A
Parameter value	N/A. This Mapping Type is used for API Adaptor only and specifies that output must be serialized according to an output class serialization mechanism (see Figure 23).

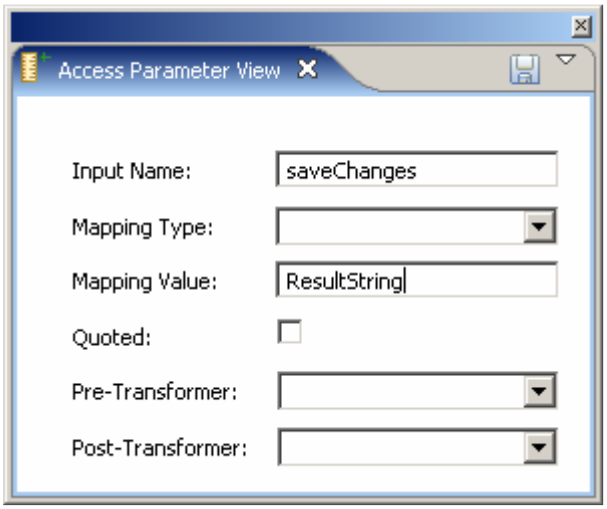

Figure 23

1.1.2.3.2 Method Type <java type>

When parameter is created Mapping Type is replaced by Qualified Class Name (including package) of the output type.

Input Name	Method Name.
Mapping value	Name for the IWP "col" element
Quoted	N/A
Parameter Value	N/A. This type is used by API Adaptor to define method type (Figure 24)

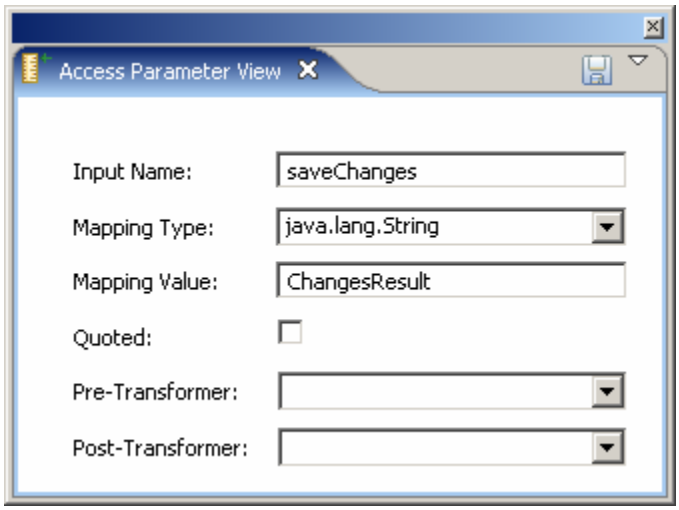

Figure 24

1.1.2.3.3 Method Type "XML"

Input Name	Method Name
Mapping value	N/A
Quoted	N/A
Parameter value	N/A. This Mapping Type is used for API Adaptor only and specifies that method output is an XML string (see Figure 25).

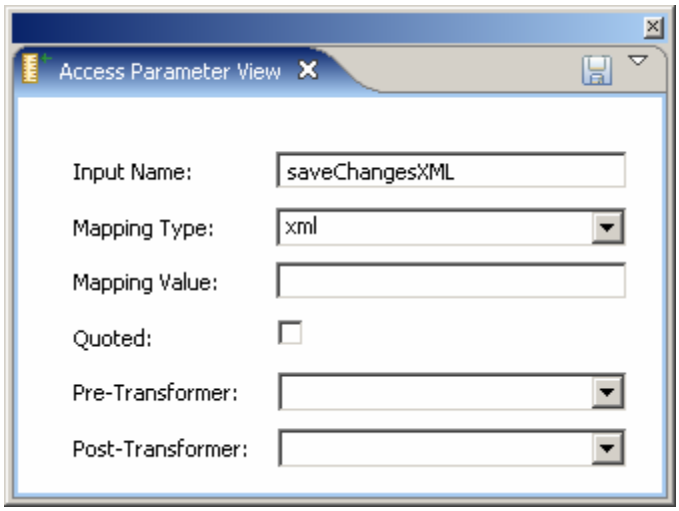

Figure 25

1.1.2.3.4 Method Type "xpath"

Input Name	Parameter Name
Mapping value	XPath expression to execute.
Quoted	N/A
Parameter value	The result of XPath expression execution against previous transactions results will be placed in the new global parameter or will replace a value of the existing one. Generated after command execution (see Figure 17).

1.1.2.3.5 Method Type "reflect"

Input Name	Parameter Name
Mapping value	New parameter value.
Quoted	N/A
Parameter value	New Parameter Value (from Mapping Value) will be set as a value of this global parameter for the future transactions and used for this one. Generated before command execution (seeFigure 26).

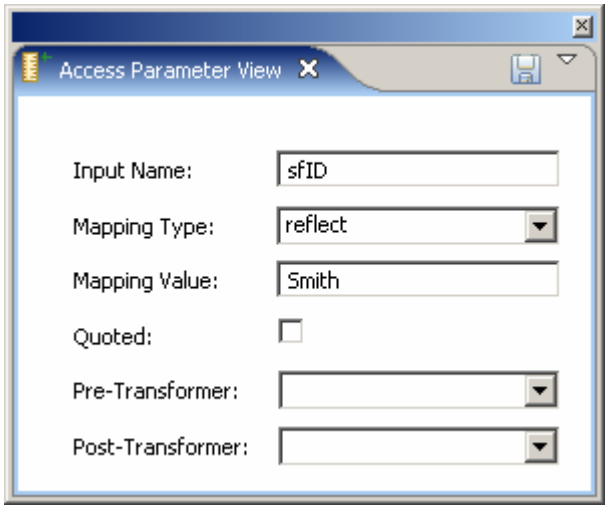

Figure 26

1.2 Dynamic Connection parameters

Each connection has 4 standard elements:

- **Driver or Type**
- **URL**
- **User Id**
- **Password**

Driver or Type element is Adaptor specific. Its syntax will be described below in each Adaptor definition. The rest 3 connection elements are self-explanatory.
Each element can be specified explicitly or parameterized. For Password element "As Reference" check box must be checked if it is parameterized. There are 2 types of parameterization - reference to Transaction flow parameter and reference to Data Map parameter.

1.2.1 Reference to Transaction Flow Parameter

Reference to Transaction Flow Parameter has the following format:

@Prameter_Name@

where Parameter_Name is a name of transaction flow parameter. The value of this parameter will be used as a value of corresponding connection element (see Figure 27).

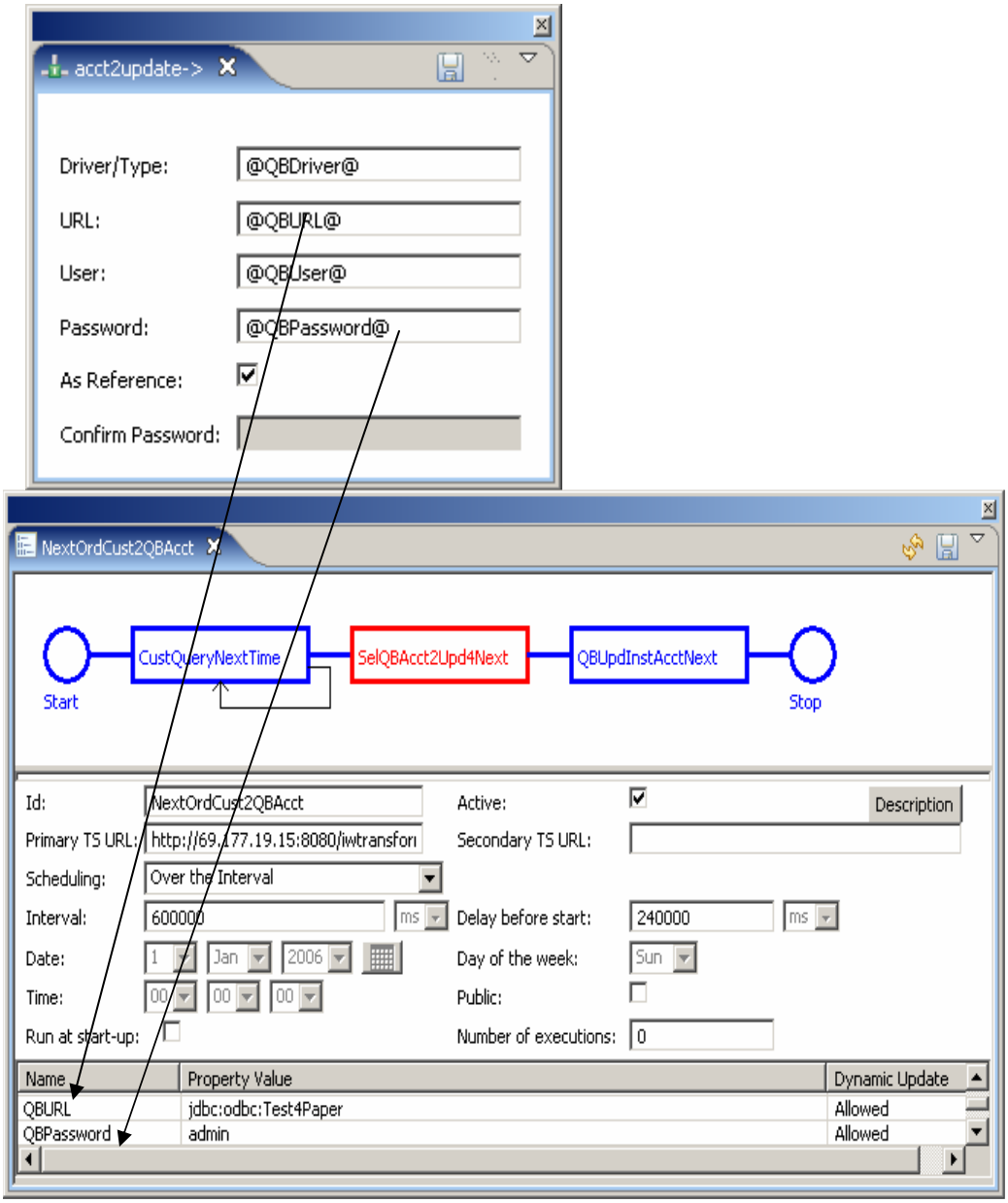
acct2update->
Driver/Type:
@QBDriver@
URL:
@QBURL@
User:
@QBUser@
Password:
@QBPassword@
As Reference:
Confirm Password:
NextOrdCust2QBAcct
Start
CustQueryNextTime
SelQBAcct2Upd4Next
QBUpdInstAcctNext
Stop
Id:
NextOrdCust2QBAcct
Active:
Description
Primary TS URL:
http://69.177.19.15:8080/iwtransfor
Secondary TS URL:
Scheduling:
Over the Interval
Interval:
600000
ms
Delay before start:
240000
ms
Date:
1
Jan
2006
Day of the week:
Sun
Time:
00
00
00
Public:
Run at start-up:
Number of executions:
0
Name
Property Value
Dynamic Update
QBURL
jdbc:odbc:Test4Paper
Allowed
QBPassword
admin
Allowed

Figure 27

1.2.2 Reference to Data Map parameter

Reference to Data Map parameter has the following format:

?Parameter_Name?

where Parameter_Name is the name of the parameter in enclosing Data Map. Data Map parameter name is defined by the "Input Name" field in the parameter view, value depends on the parameter definition (see above 1.1.2, see Figure 28).

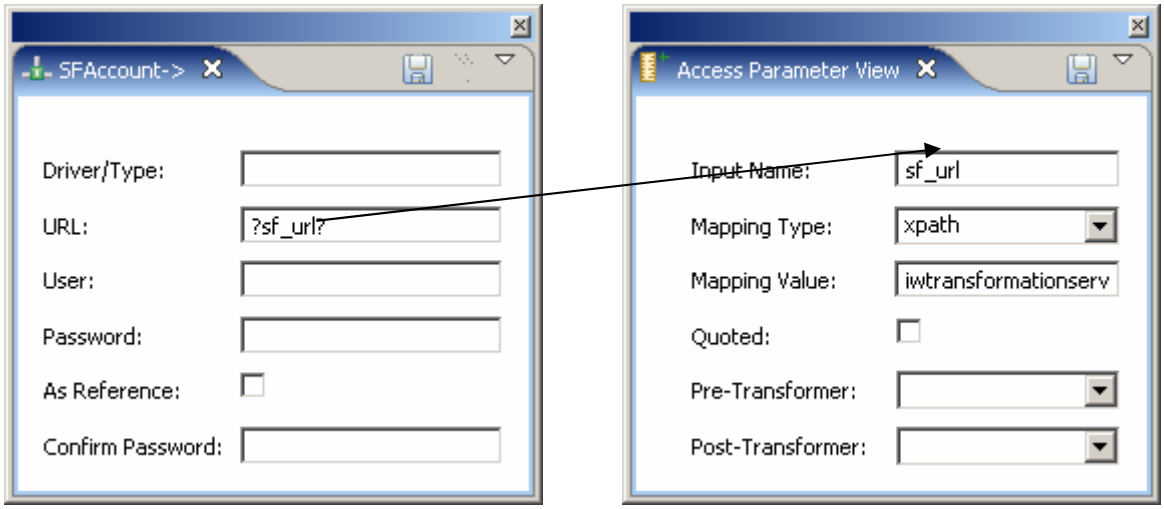

Figure 28

1.3 Dynamic command parameters

Every command in Data Map can have dynamic elements. There are 2 types of dynamic elements: substitute parameters and global variables. These elements are applicable to both types of access – procedural and dynamic. For procedures they are explicitly specified in the command, for dynamic they can be included into corresponding transformer (pre-transformer).

1.3.1 Substitute parameters

Substitute parameters are used to replace question marks ("?" – reserved substitute character; question marks that are parts of native command syntax, e.g. XML declaration, are ignored) with the parameter with the corresponding 0-based sequence number. Parameter_0 replaces first question mark, Parameter_1 – second etc.(seeFigure 29) Value of parameter that will replace question mark depends on the parameter definition (see above 1.1.2.1).

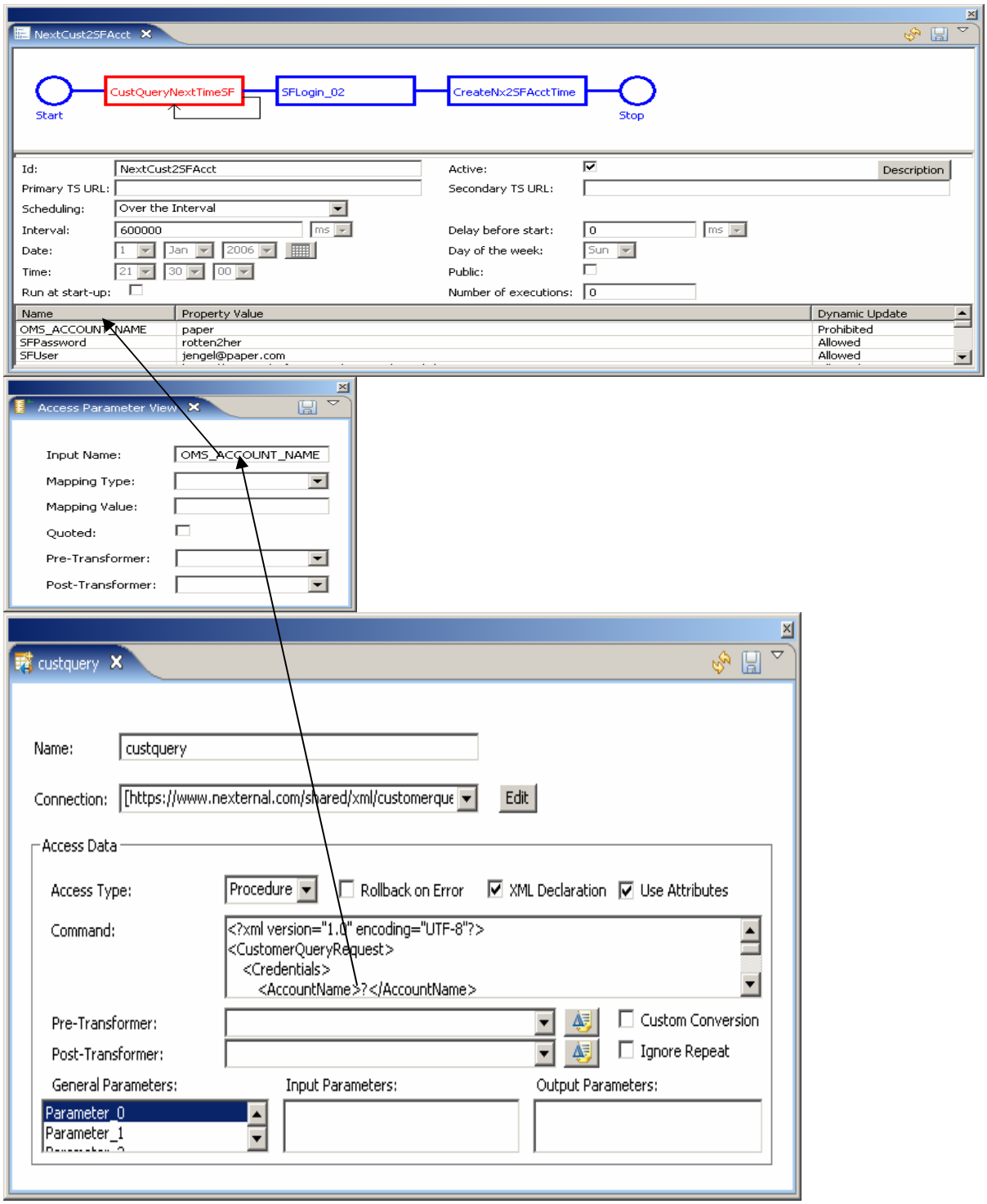
NextCust2SFAcct
Start
CustQueryNextTimeSF
SFLogin_02
CreateNx2SFAcctTime
Stop
Id:
NextCust2SFAcct
Active:
Description
Primary TS URL:
Secondary TS URL:
Scheduling:
Over the Interval
Interval:
600000
ms
Delay before start:
0
ms
Date:
1
Jan
2006
Day of the week:
Sun
Time:
21
30
00
Public:
Run at start-up:
Number of executions:
0
Name
Property Value
Dynamic Update
OMS_ACCOUNT_NAME
paper
Prohibited
SFPassword
rotten2her
Allowed
SFUser
jengel@paper.com
Allowed
Access Parameter View
Input Name:
OMS_ACCOUNT_NAME
Mapping Type:
Mapping Value:
Quoted:
Pre-Transformer:
Post-Transformer:
custquery
Name:
custquery
Connection:
[https://www.nexternal.com/shared/xml/customerque
Edit
Access Data
Access Type:
Procedure
Rollback on Error
XML Declaration
Use Attributes
Command:
<?xml version="1.0" encoding="UTF-8"?>
<CustomerQueryRequest>
<Credentials>
<AccountName>?</AccountName>
Pre-Transformer:
Custom Conversion
Post-Transformer:
Ignore Repeat
General Parameters:
Input Parameters:
Output Parameters:
Parameter_0
Parameter_1

Figure 29

1.3.2 Global variables

All global variables have the following format:

%variable_name%

and will be replaced by their values before command execution (see Figure 30). Here is the list of global variables:

Variable	Description
%iw_password%	The value of connection element Password from enclosing Data Map connection.
%iw_user%	The value of connection element User from enclosing Data Map connection.
%current_time%	Current time of transaction execution in default locale in JDBC time escape format `hh:mm:ss`
%current_date%	Current date of transaction execution in default locale in JDBC date escape format yyyy-mm-dd
%current_timestamp%	Current timestamp of transaction execution in default locale in JDBC timestamp escape format `yyyy-mm-dd hh:mm:ss.fffffffff`
%start_time%	Time part of QueryStartTime timestamp (transaction flow built-in parameter) in default locale in JDBC time escape format `hh:mm:ss`
%start_date%	Date part of QueryStartTime timestamp (transaction flow built-in parameter) in default locale in JDBC time escape format yyyy-mm-dd
%start_timestamp%	QueryStartTime timestamp (transaction flow built-in parameter) in default locale in JDBC time escape format `yyyy-mm-dd hh:mm:ss.fffffffff`
%loop_index%	Loop counter (0-based) for loops in transaction flow. Values are from 0 to n-1 where n is the number of iterations in the loop.
%loop_index_1%	Loop counter (1-based) for loops in transaction flow. Values are from 1 to n where n is the number of iterations in the loop.
%current_date_us%	Current date of transaction execution in US locale format mm/dd/yyyy
%start_time_m%	Current time plus 1 minute.

%start_time_1%	Time part of QueryStartTime timestamp (transaction flow built-in parameter) in default locale in JDBC time escape format hh:mm:ss increased in 1 ms.
%start_date_us%	Date part of QueryStartTime timestamp (transaction flow built-in parameter) in US locale format mm/dd/yyyy
%start_timestamp_1%	QueryStartTime timestamp (transaction flow built-in parameter) in default locale in JDBC time escape format `yyyy-mm-dd hh:mm:ss.fffffffff` increased in 1ms.
%start_timestamp_z%	QueryStartTime timestamp (transaction flow built-in parameter) in default locale in JDBC time escape format `yyyy-mm-dd hh:mm:ss.fffffffff` decreased by time zone shift value from Company configuration.
%return_string%	The value of the parameter of the Return_String parameter.
%stop_scheduler%	Stop scheduled transactions.
%session_id%	Unique Interweave Session id.

custquery

Name: custquery

Connection: [https://www.nexternal.com/shared/xml/customerquery.asp] Edit

Access Data

Access Type: Procedure Rollback on Error XML Declaration Use Attributes

Command:

```
<DateTime>
    <Date>%start_date%</Date>
    <Time>%start_time%</Time>
```

Pre-Transformer: Custom Conversion

Post-Transformer: Ignore Repeat

General Parameters: Input Parameters: Output Parameters:

Parameter_0
Parameter_1

Figure 30

1.3.3 Transaction Flow Counter

There is a special parameter that can be used in a Data Map – Transaction Flow Counter. Its format is represented by the reserved “TransactionFlowCounter” name in the Data Map parameter “Input Name” element. This parameter is incremented by one after every transaction flow execution. It can be set initially during transaction flow design and changed dynamically via WEB interface (see Figure 31).

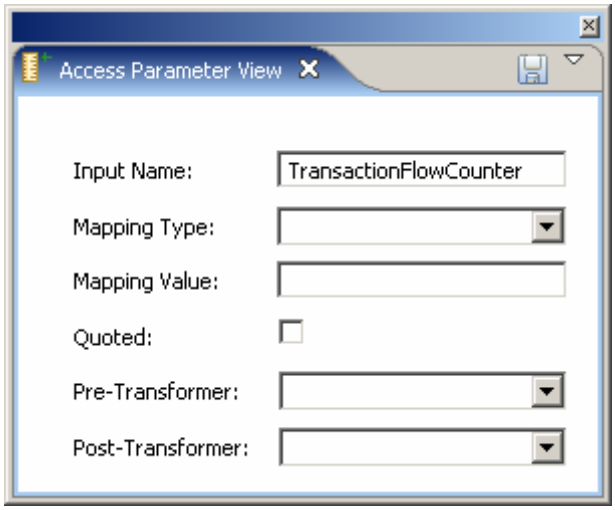

Figure 31

2 IW Adaptors

2.1 JDBC Adaptor

This adaptor executes SQL queries against any database that has JDBC driver available. Driver and URL connection parameters are specific to the JDBC driver used and must be obtained from the vendor's or supplier's driver documentation. For example, calling the Sun JDBC/ODBC bridge would require you to identify the driver – sun.jdbc.odbc.JdbcOdbcDriver, a DSN (Data Source Name)- jdbc:odbc:mydata, and user credentials to access the data – username: root, password: foobar.

2.1.1 Asynchronous JDBC Adaptor

This adaptor executes queries asynchronously (concurrently).

Class name: *com.interweave.adapter.database.IWSqlBase*

2.1.2 Synchronous JDBC Adaptor

This adaptor executes queries synchronously (sequentially).

Class name: *com.interweave.adapter.database.IWSqlSync*

2.2 HTTP Adaptor

This adaptor executes HTTP request “PUT” against any HTTP server and reads the returned stream.

.
Class name: *com.interweave.adapter.http.IWHttpBaseAdaptor*.

Connection parameters:

- Type string defines content type and authorization of HTTP request. Default content type is “text/plain” and there are no defaults for Basic Authorization parameters. Format of the Driver/Type element is the following:

  <content type> HTTP_BASIC_AUTH:<user>:<password>

  where content *type* is a MIME type of the request and *user* and *password* are Basic Authorization credentials (see Figure 32).

  Space is the delimiter between content type and “HTTP_BASIC_AUTH:” token. They can be entered in any order. Both parts are optional.
  For the content type *application/x-www-form-urlencoded* special format can be used:

  application/x-www-form-urlencoded:<MODE>

  where MODE can be:

  - **NOENCODE** – no url-encoding of output stream, url-decoding of input stream;
  - **NODECODE** – url-encoding of output stream, no url-decoding of input stream;
  - **NONE** – neither url-encoding nor url-decoding should be performed (see Figure 33)

- URL string defines HTTP server URL

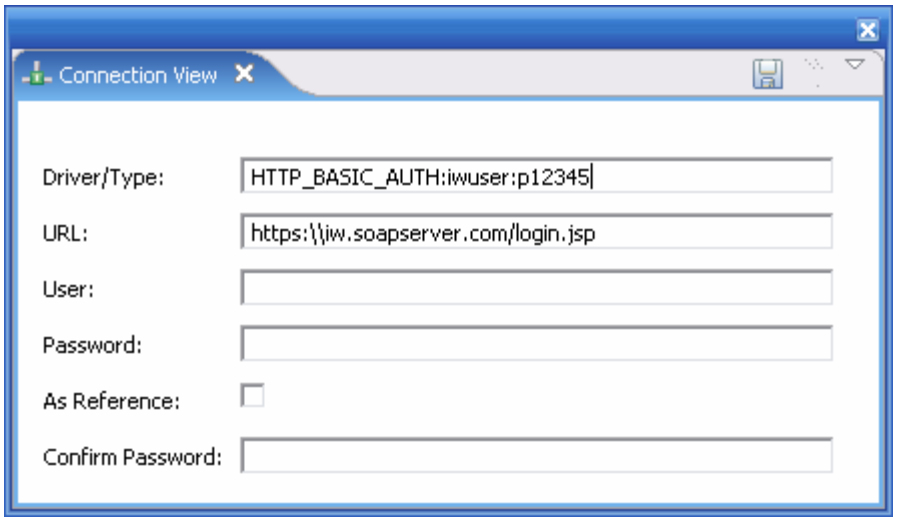

Figure 32

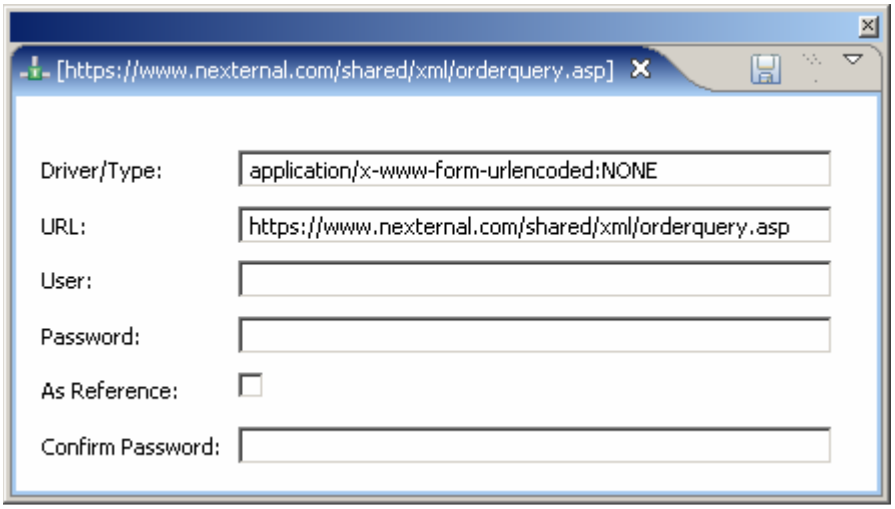

Figure 33

2.3 XML over HTTP Adaptors

These adaptors executes HTTP request "PUT" against any HTTP server and reads the returned stream. Both input and output streams should be in XML format.

Connection parameters are the same as for HTTP Adaptor (see 2.2) with one exception: default content type is "text/xml; charset=utf-8". User and Password connection elements can be used if protocol requires.

2.3.1 Base XML Adaptor

This adaptor converts incoming XML into IWP format. All “leaves” of incoming XML tree will be represented as “col” elements of IWP with corresponding names. Hierarchical tree nodes will not be preserved.

Class Name: *com.interweave.adapter.http.IWXMLBaseAdaptor*

2.3.2 Hierarchical XML adaptor

This adaptor converts incoming XML into HIWP format. All “leaves” of incoming XML tree will be represented as “col” elements of HIWP with corresponding names. Hierarchical tree nodes will be represented as “rowset” elements with corresponding names. Hierarchical tree relationships will be preserved.

Class Name: *com.interweave.adapter.http.IWXMLHierarchicalAdaptor*

2.4 SOAP over HTTP Adaptors

These adaptors executes HTTP request “PUT” against any SOAP server and reads the returned stream. Both input and output streams should be in SOAP format.

Connection parameters are the same as for HTTP Adaptor (see 2.2) with one addition: Type parameter can have a SOAP action element. The complete format of Type element will be:

<content type> HTTP_BASIC_AUTH:<user>:<password> HTTP_SOAP_ACTION:<SOAP_Action_URI>

where SOAP_Action_URI is the URI of the SOAP action.

Space is the delimiter between parts. They can be entered in any order. All parts are optional.

Default URI for SOAP action is “SOAPAction”; default content type is “text/xml; charset=utf-8” (see Figure 34).

User and Password connection elements can be used if protocol requires.

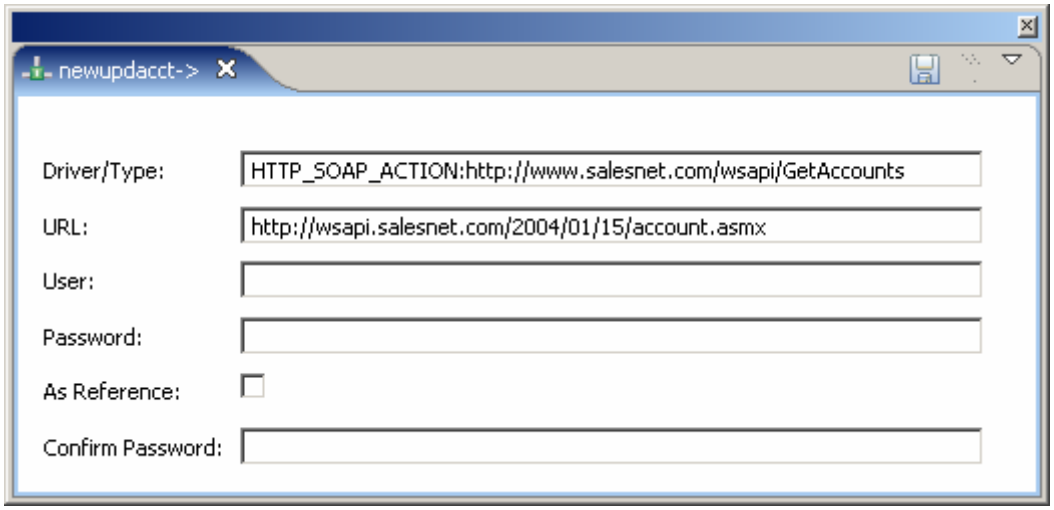

Figure 34

2.4.1 Base SOAP Adaptor

This adaptor converts incoming SOAP message into IWP format. All "leaves" of incoming SOAP XML tree will be represented as "col" elements of IWP with corresponding names. Hierarchical tree nodes will not be preserved.

Class Name: *com.interweave.adapter.http.IWSoapBaseAdaptor*

2.4.2 Hierarchical SOAP adaptor

This adaptor converts incoming SOAP message into HIWP format. All "leaves" of incoming SOAP XML tree will be represented as "col" elements of HIWP with corresponding names. Hierarchical tree nodes will be represented as "rowset" elements with corresponding names. Hierarchical tree relationships will be preserved.

Class Name: *com.interweave.adapter.http.IWSoapHierarchicalAdaptor*

2.5 File Adaptors

These adaptors perform basic file operations against files in a file system. User and Password connection elements can be used if protocol requires. A URL connection element defines file path or URL depending on the protocol used. Type connection element contains file type and command to execute. The general format will be:

<TYPE>;<COMMAND>:<PARAMETER1>:<PARAMETER2>…

where TYPE is a file type and COMMAND is a command to execute; command may optionally have parameters (colon is a delimiter between them).

Supported file types:

- **TXT** – plain text file;
- **TXTL** – text file that consists of set of lines;
- **NVP** – file containing name-value pairs; file format is identical to a properties file format in Java;
- **CSV** – text file containing coma separated values organized as lines of text;
- **CSVH** – text file containing coma separated values organized as lines of text where first line contains the header – set of names for the rest of values;
- **XML** – file containing XML document.

Supported commands:

- **READ** – read the whole file;
- **READ**:pos1:po2 – read file from position pos1 inclusive to position pos2 exclusive; the meaning of position in the file – see below;
- **WRITE** – create a new file or files – see below;
- **UPDATE** – overwrite existing file;
- **UPDATE**:pos1:pos2 - overwrite part of the file from position pos1 inclusive to position pos2 exclusive; the meaning of position in the file – see below;
- **INSERT**:pos1 – insert data to a file at position pos1; the meaning of position in the file – see below;
- **APPEND** – append data to a file;
- **REANAME** – rename file;
- **MOVE** – move file to a new location;
- **DELETE** – delete file.

Position meaning:

- **TXT** – character sequence number;
- **TXTL, CSV, CSVH** – line sequence number.

WRITE, **APPEND**, and **UPDATE** (with no parameters) commands for line base file types (**TXTL**, **CVS**, and **CVSH**) can process multiple files in the same directory. In this case URL connection element must define a directory and file names must be specified in the input data as separate lines (no other information in those lines is allowed). These lines must have the following format:

%%__FILE_NAME__%%

where FILE_NAME is the name of the file being written to.

Important! All commands with position parameters are not applicable to **XML** and **NVP** file types; **APPEND** command is not applicable to **XML** file type.

2.5.1 Base File Adaptor

This adaptor performs basic file operations against text files in a local file system.

Class Name: *com.interweave.adapter.filesystem. IWFileBaseAdaptor*

This Adaptor supports group operations where file name has format "*.*ext*" ("*ext*" – any extension). In this case corresponding command will be applied to all matching files.

2.5.2 Base FTP Adaptor

This adaptor performs basic file operations against text files on remote server via FTP protocol..

Class Name: *com.interweave.adapter.filesystem. IWFtpBaseAdaptor*

URL connection element must follow the regular FTP URL pattern, but "userinfo" part will be ignored if present. User and Password connection elements will be used instead. URL can also have a query part of the following format:

*ftp://<host>/<path>**?type=<file_type>***

where file_type can be:

- ASCII – file being transferred should be treated as ASCII; this is the default file type
- BINARY – file being transferred should be treated as binary;
- IMAGE – same as binary.

This Adaptor does not support group operations (explicit file name must be used). It also does not support multiple file definitions in the incoming data (see *%%__FINE_NAME__%%* format above). This functionality can be implemented using transaction looping feature.

2.6 API Adaptors

These adaptors are used to invoke methods either directly on integrated objects API or on the IW plug-ins that will control integrated object API. Usage of connection elements depends on the adaptor type.

2.6.1 Local API Adaptor

This adaptor can be used to call methods on objects in the same JVM as transformation server.

Class Name: *com.interweave.adapter.api. IWLocalAPIBaseAdaptor*

In most cases this will be IW plug-ins unless integrated application is also a WEB-application running in the same servlet container. It is recommended to call static methods when using this adaptor. For non-static methods parent class must have a constructor without parameters and for each call a new instance of the object will be created. So state can be preserved only via static variables that have static setter methods.

Full method signature – package.class.method(Value0, Value1, ...) goes to the Data Map command; input parameters go into Data Map Input Parameters (Input Name contains full method name; Mapping Value contains parameter index from 0; Mapping Type contains qualified class name – see 1.1.2.2). Optional Output Parameter describes method output (Input Name contains method name, Mapping Value contains optional IWP "col" name; Mapping Type contains output type - only XML is currently supported; see 0).

Result is represented in IWP as data tag per method. List of XML strings is not supported. Output as List interface implementations and XML create multiple rows in IWP; each row for the regular Java class and List has one only "col" element; Map interface implementations can have multiple "col" elements; XML and List of Maps can have multiple "col" and "row" elements.

2.7 E-mail Adaptors

These adaptors perform basic e-mail operations send and receive. User and Password connection elements can be used if authorization is required by E-mail service provider. A URL connection element defines host and optional port of the E-mail server. Type connection element contains e-mail protocol and command to execute. The general format will be:

<PROTOCOL>:<COMMAND>…

where supported protocols are:

- SMTP – to send e-mails;
- POP3 – to receive e-mails;
- IMAP – to receive web mails.

Protocols POP3 and IMAP support the following commands:

- READ – read all e-mails except marked as deleted;
- READALL – read all e-mails;
- READNEW – read only recent e-mails;
- MOVE – read all e-mails and delete them from the server.

.

2.7.1 Base E-mail Adaptor

This adaptor performs basic e-mail operations.

Class Name: *com.interweave.adapter.email.IWEmailBaseAdaptor*